



UNIVERSIDAD DE DAGUPAN

SCHOOL OF INFORMATION TECHNOLOGY EDUCATION

ITP10 | EVENT-DRIVEN PROGRAMMING

MIDTERM | LABORATORY ACTIVITY 2

Name : Samson, Laurence D.
Year, Course & blk : 31-ITE-01
Subject : ITP 10 - Event Driven Programming
Date : October 2, 2027

SCORE:

Laboratory Activity #2: Secure Dynamic Task Manager

Objective: After completing the laboratory activity, students should be able to create, insert, traverse, modify, replace, and remove DOM elements programmatically; use event delegation for dynamically generated controls; batch DOM insertion with DocumentFragment; and protect dynamic content by using safe DOM APIs instead of HTML injection.

Activity Overview: Students will develop a Secure Dynamic Task Manager. Tasks must be created entirely through JavaScript, added to the live DOM, edited, completed, and removed without page reloads. The application must use traversal and event delegation so controls continue to work for tasks created after the page has loaded.

Required Project Structure

```
secure-task-manager/  
├── index.html  
├── css/  
│   └── style.css  
└── js/  
    └── app.js
```

Application Requirements

1. Initial Application State

- The task list must be empty on initial page load.
- The initial summary values must be Total Tasks: 0, Pending: 0, and Completed: 0.
- No task-item elements may be hard-coded in index.html. All task items must be created through JavaScript.

2. Required JavaScript Functions

- createTaskElement(taskText, taskId) — create and return one <li class="task-item"> element with its text and Complete, Edit, and Remove buttons. The function must not attach the item to #taskList by itself.
- addTask(taskText) — validate the text, create a unique task ID, call createTaskElement(), append the task to #taskList, clear the input, clear any error message, and update the counts.
- toggleTaskComplete(taskItem) — toggle the completed state using classList and update taskItem.dataset.state between "pending" and "completed".
- beginTaskEdit(taskItem) — replace the visible task text span with an <input class="edit-input"> containing the current text and change the Edit button text to Save.
- saveTaskEdit(taskItem) — validate the edit input, create a new using textContent, replace the edit input, and change Save back to Edit.
- removeTask(taskItem) — remove the selected task from the DOM using remove() and update the counts.
- updateTaskCounts() — calculate and display total, pending, and completed task counts from the current DOM.
- handleTaskListClick(event) — the single delegated click handler for Complete, Edit/Save, and Remove actions.
- loadSampleTasks() — create the three required sample tasks using a DocumentFragment and append the fragment to #taskList once.

3. Required Task Structure

```
<li class="task-item" data-task-id="task-1" data-state="pending">  
  <span class="task-text">Example task</span>  
  <button class="complete-btn">Complete</button>  
  <button class="edit-btn">Edit</button>  
  <button class="remove-btn">Remove</button>  
</li>
```

- Every new task must use a unique data-task-id value beginning with task-.
- Every new task must start with data-state="pending".
- The task text span, buttons, and edit input must be created with document.createElement().

- Assign user-provided task text only through `textContent`. Do not construct task markup using string concatenation and `innerHTML`.

4. Event Delegation and Traversal Rules

- Attach exactly one click event listener to `#taskList` for task-level Complete, Edit/Save, and Remove actions.
- Inside `handleTaskListClick(event)`, use `event.target.matches()` or `classList.contains()` to identify the clicked action button.
- Use `event.target.closest(".task-item")` to locate the task that owns the clicked button.
- Do not attach separate click listeners to every dynamically created task button.
- Use `taskItem.remove()` for deletion. Do not rebuild the entire list just to remove one task.

5. Security and Performance Rules

- The exact message "Task cannot be empty" must be displayed in `#taskMessage` when Add Task or Save is attempted with blank/whitespace-only text.
- Never use `innerHTML`, `insertAdjacentHTML()`, `document.write()`, or inline `onclick` attributes for task creation or task actions.
- Use `textContent` for every user-entered or edited task value.
- `loadSampleTasks()` must use `document.createDocumentFragment()` and append the fragment to `#taskList` only once after all three sample task elements are created.
- Use `classList.toggle("completed")` to manage the visual completed state.

6. Required Sample Tasks

- Review DOM selectors
- Practice `createElement`
- Study event delegation

7. User Interface and Required IDs

Interface Element	Required ID / Class	Required Text / Value	Notes
Application title	—	Secure Dynamic Task Manager	Visible title
Task input	<code>taskInput</code>	<code>placeholder="Enter a task"</code>	Text input
Add button	<code>addTaskBtn</code>	Add Task	Button
Load samples button	<code>loadSamplesBtn</code>	Load Sample Tasks	Button
Task list	<code>taskList</code>	—	Use <code></code> or <code></code>
Task item	<code>.task-item</code>	—	One per task
Task text	<code>.task-text</code>	—	Created with <code>textContent</code>
Complete button	<code>.complete-btn</code>	Complete	Dynamic button
Edit button	<code>.edit-btn</code>	Edit / Save	Dynamic button
Remove button	<code>.remove-btn</code>	Remove	Dynamic button
Edit field	<code>.edit-input</code>	—	Exists only while editing
Validation message	<code>taskMessage</code>	Task cannot be empty	Shown on invalid add/save
Total count	<code>totalCount</code>	0	Summary
Pending count	<code>pendingCount</code>	0	Summary
Completed count	<code>completedCount</code>	0	Summary

Expected Behavior

- Adding a valid task immediately creates and displays a new task-item without reloading the page.
- Adding HTML-like or script-like text displays the characters literally; no HTML element or script is created from the task text.
- Complete toggles the completed class and data-state value; clicking Complete again returns the task to pending.
- Edit replaces only the selected task text span with an edit input. Save safely restores a task-text span with the edited text.
- Remove deletes only the selected task.
- All dynamically created buttons work through the single delegated listener on `#taskList`.
- Load Sample Tasks adds exactly the three required sample tasks through a `DocumentFragment`.
- Summary counts always match the current DOM state.

Automated-Grading Compatibility

- Do not rename required files, function names, IDs, classes, button labels, data attributes, sample task text, or required messages.
- Do not hard-code task counts. `updateTaskCounts()` must calculate from the current task elements and their data-state values.

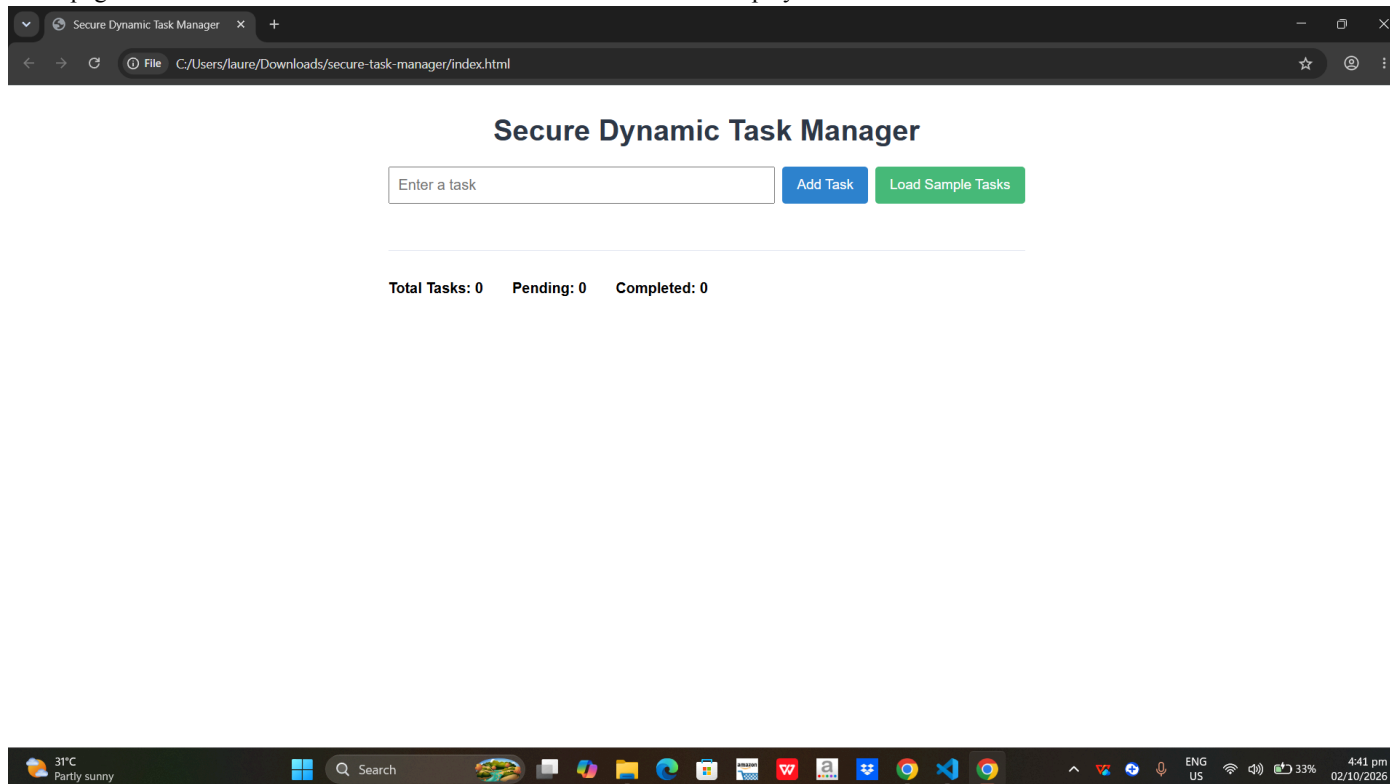
- Do not hard-code task action behavior for specific task IDs. Delegated actions must work for any task-item created by the application.
- Do not use innerHTML or inline event-handler attributes.
- Students may design the interface freely as long as the required selectors and behavior remain intact.

General Procedure

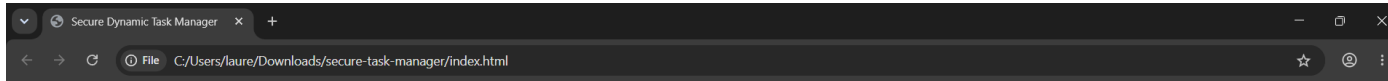
1. Create the required folders and files.
2. Build the static controls and empty task-list container in index.html using the exact IDs.
3. Create CSS for task items, completed state, buttons, edit input, validation message, and summary counters.
4. Select and store the task input, buttons, task list, message, and count elements in app.js.
5. Implement createTaskElement() using createElement(), textContent, classList, and dataset.
6. Implement addTask(), validation, unique task IDs, and count updates.
7. Attach one delegated click listener to #taskList and implement handleTaskListClick() using matches()/class checks and closest().
8. Implement completion, edit/save, and removal operations.
9. Implement loadSampleTasks() using a DocumentFragment.
10. Test safe content handling, dynamic actions, traversal, event delegation, sample loading, and all count values before submission.

Testing Requirements

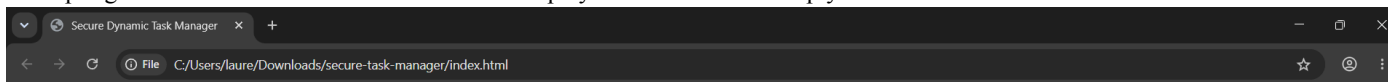
1. Initial page load contains zero .task-item elements and all three counts display 0.



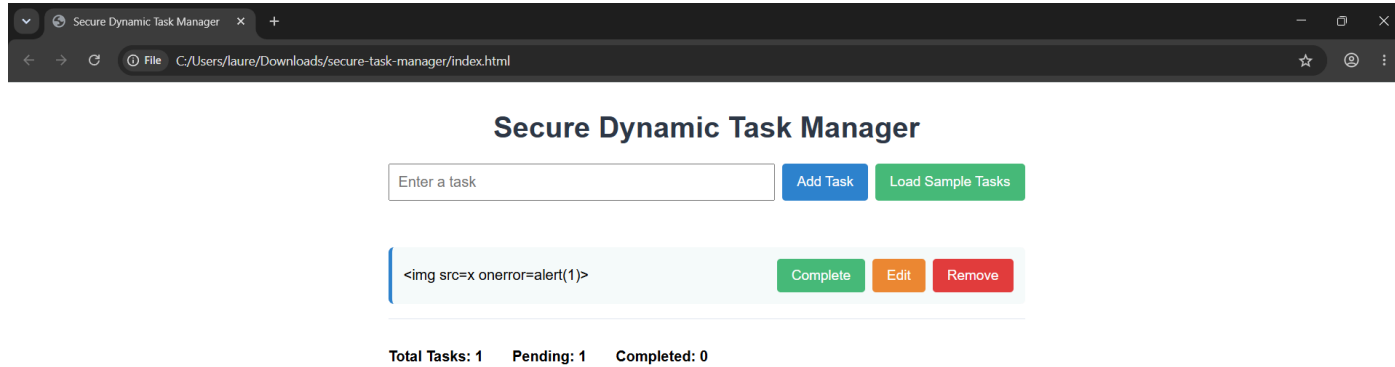
2. Adding "Finish Module 4" creates one pending .task-item and updates Total to 1 and Pending to 1.



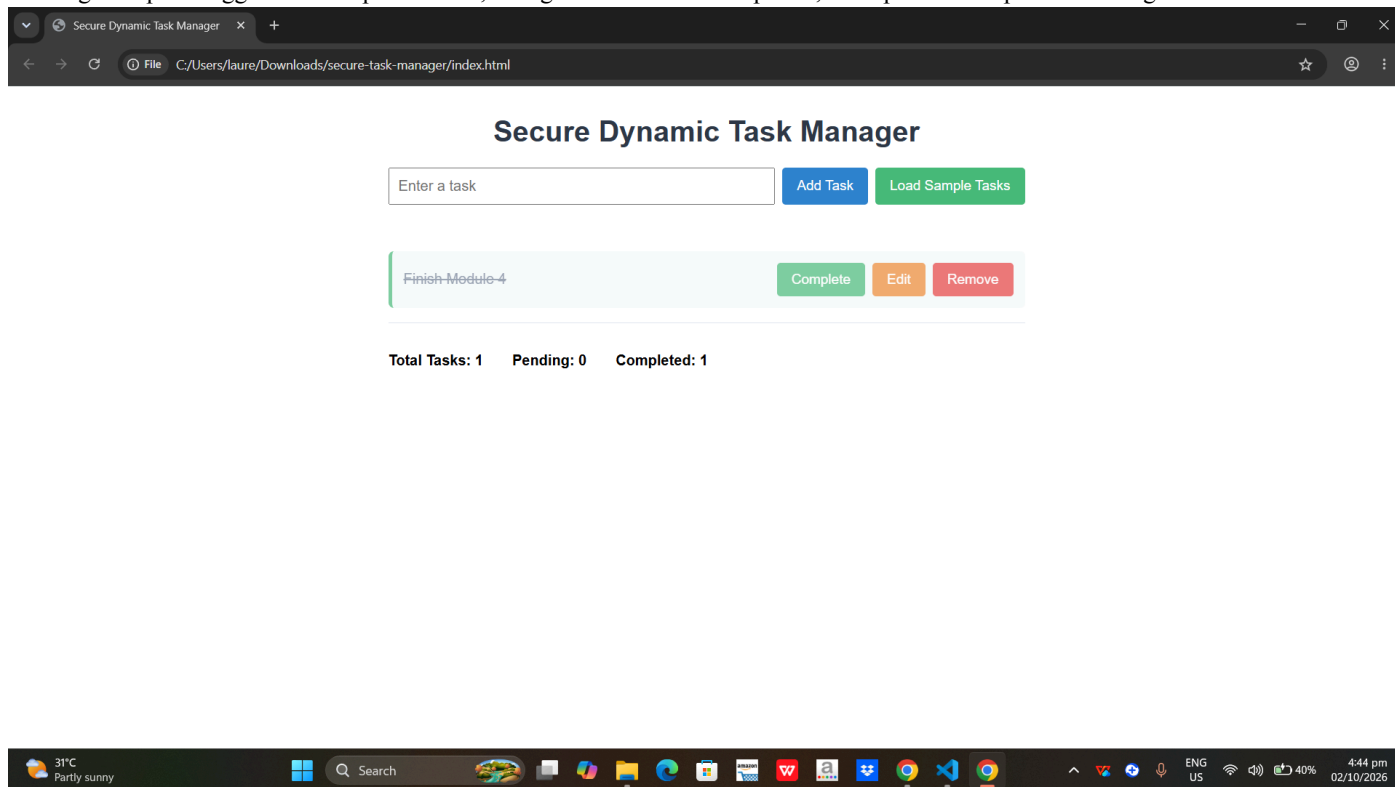
3. Attempting to add a blank task creates no task and displays "Task cannot be empty".



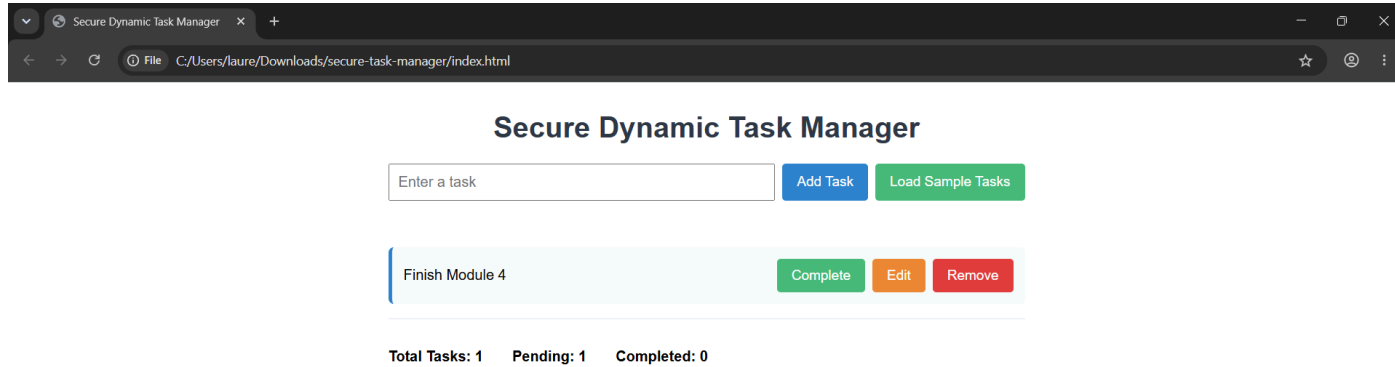
4. Adding the text `<img src=x onerror=alert(1)>` displays that string as text and does not create an img element.



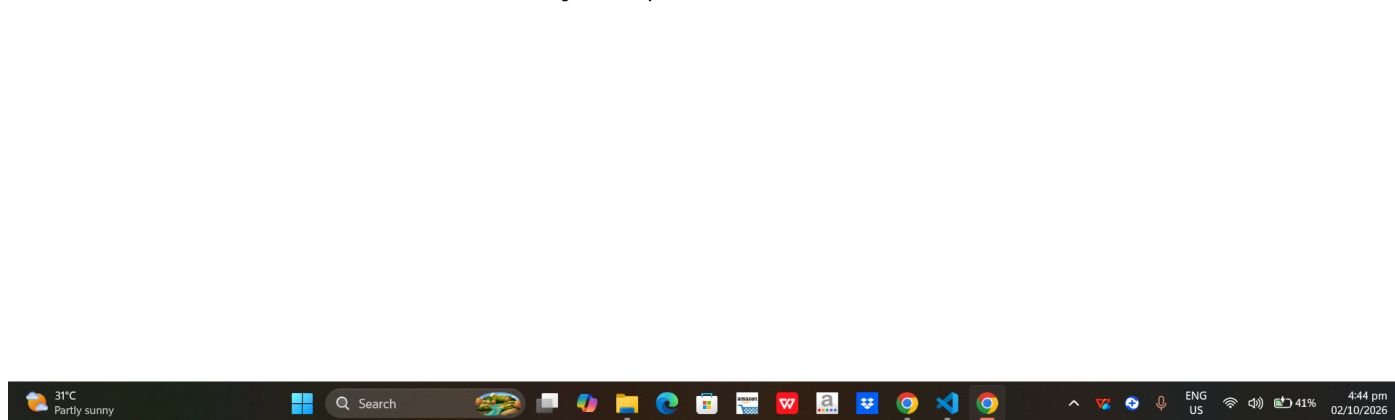
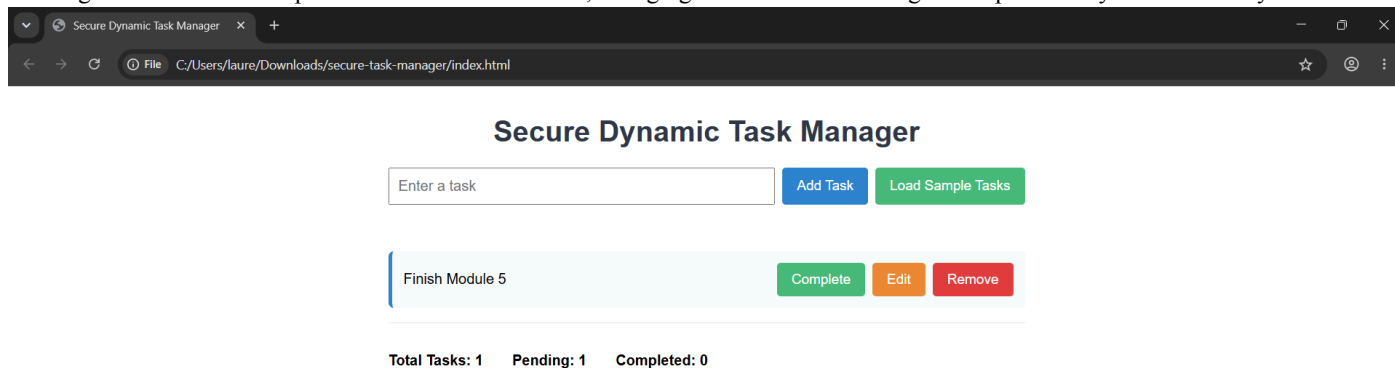
5. Clicking Complete toggles the completed class, changes data-state to completed, and updates Completed/Pending counts.



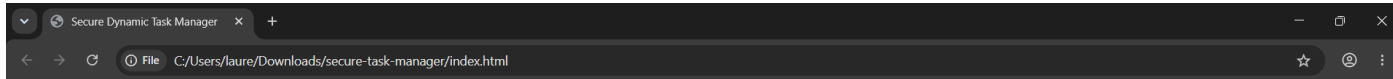
6. Clicking Complete again returns the task to pending and restores the counts.



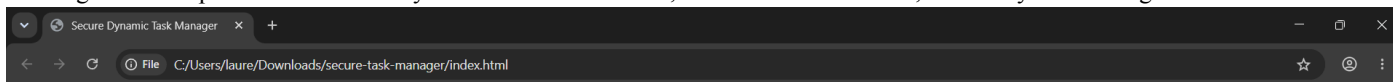
7. Clicking Edit creates .edit-input with the current task text; changing the value and clicking Save updates only that task safely.



8. Clicking Remove deletes only the selected task and updates all counts.



9. Clicking Load Sample Tasks adds exactly Review DOM selectors, Practice createElement, and Study event delegation.



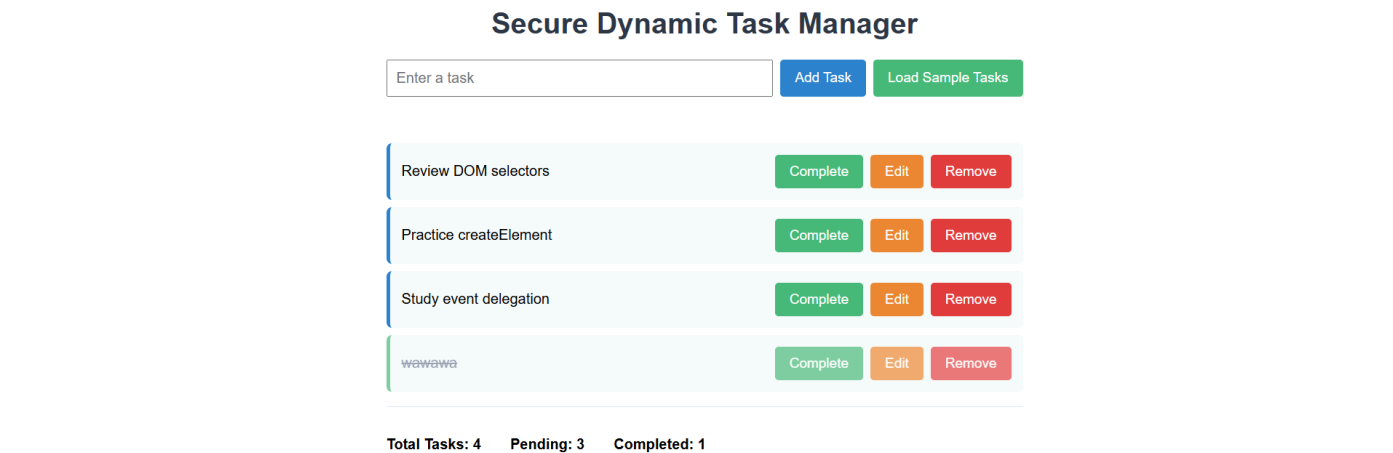
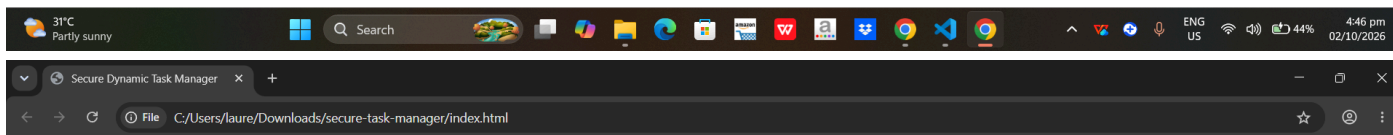
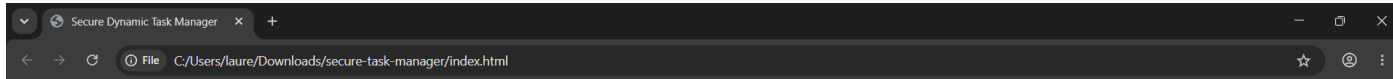
Secure Dynamic Task Manager

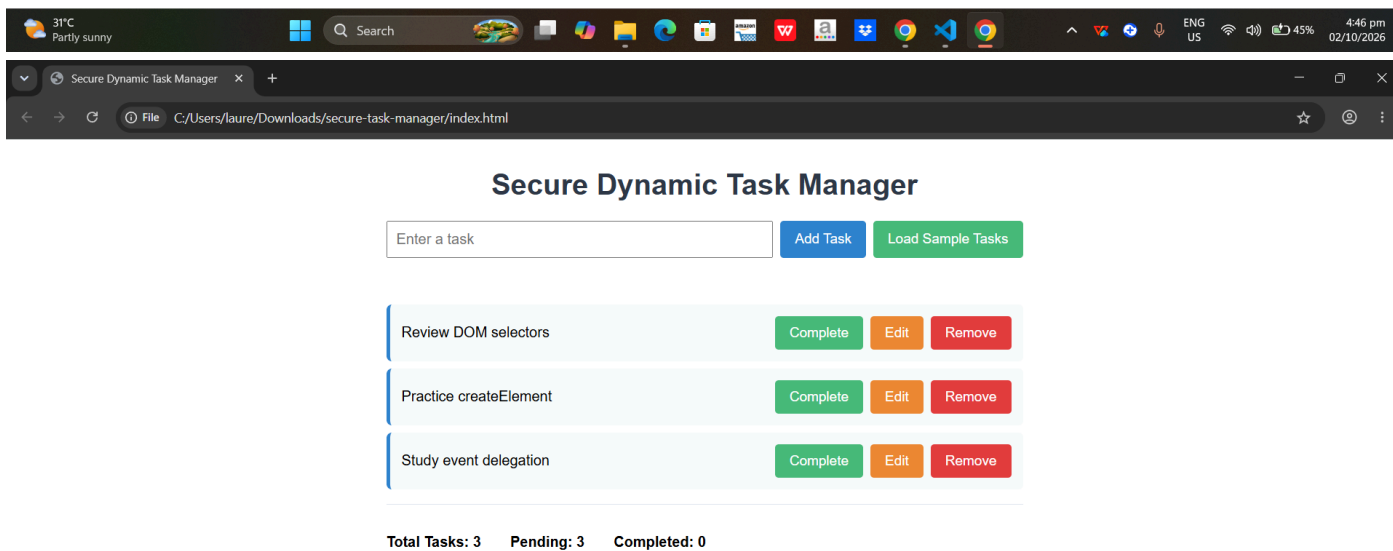
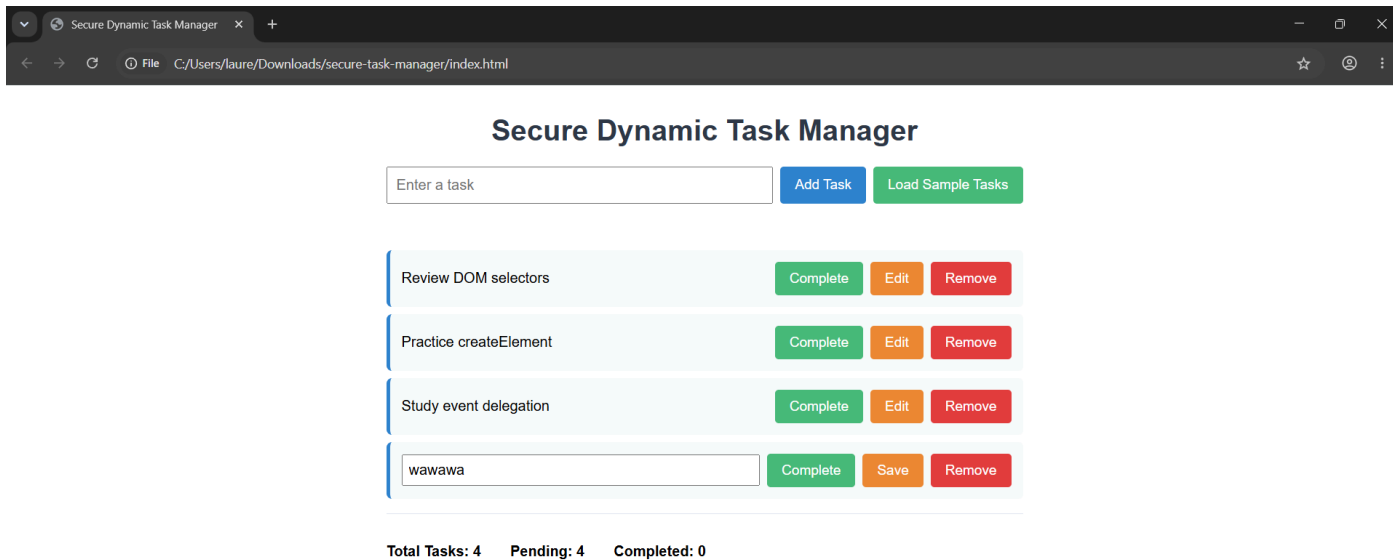
Enter a task	Add Task	Load Sample Tasks
Review DOM selectors	Complete	Edit Remove
Practice createElement	Complete	Edit Remove
Study event delegation	Complete	Edit Remove

Total Tasks: 3 Pending: 3 Completed: 0



10. Complete, Edit/Save, and Remove continue to work for tasks created after initial page load through event delegation.





Submission Requirements

- Submit the complete secure-task-manager project/repository with the required folder structure.
- Make sure the browser console contains no JavaScript errors.
- Include the screenshots or explanations required by the instructor/classroom submission page.

Grading Rubric

Criterion	Excellent	Proficient	Developing	Beginning
Application Functionality (30 points)	All required features work correctly. (27–30)	Most features work with minor errors. (22–26)	Some features work, but several errors are present. (16–21)	The application is incomplete or does not function properly. (0–15)
Modular Organization (25 points)	Code is properly divided into data, utility, display, and main modules with correct imports and exports. 23–25	Modules are properly used with minor organizational problems. 18–22	Modules are present, but responsibilities or connections are unclear. 13–17	Code is mainly placed in one file, or modules do not work. 0–12
Use of JavaScript Concepts (20 points)	Correctly uses functions, arrays, objects, callbacks, array methods, and destructuring. 18–20	Uses most required concepts correctly. 14–17	Uses some concepts but with noticeable errors. 10–13	Required JavaScript concepts are mostly missing or incorrectly applied. 0–9
User Interface and Output (10 points)	Interface is clear, organized, readable, and displays complete information. 9–10	Interface is usable with minor presentation issues. 7–8	Interface or displayed information is incomplete. 5–6	Interface is difficult to use or output is unclear. 0–4
Code Quality (10 points)	Code is readable, properly indented, meaningfully named, and free from unnecessary repetition. 9–10	Code is generally readable with minor inconsistencies. 7–8	Code has inconsistent formatting or unnecessary repetition. 5–6	Code is difficult to understand or poorly organized. 0–4
Testing and Submission (5 points)	All required files, screenshots, explanations, and test results are complete. 5	One minor requirement is missing. 4	Several submission requirements are incomplete. 2–3	Submission is substantially incomplete. 0–1
Total				100 points

PREPARED BY:

ROCHELLE S. LANTO, MIT & JELSON V. LANTO, MIT
SITE, FACULTY